

Titan Tiny Homes — Proposed Security Hardening

Prepared: 2026-05-17 | **Prepared by:** GLV Dev Agent **Client:** Titan Tiny Homes | **Domain:** titantinyhomes.ca **Hosting:** SiteGround GrowBig | **Stack:** React SPA (Lovable) via GitHub → SiteGround FTP **Client status:** Active (favour engagement — not currently paying; deprioritize effort accordingly)

> Note: Titan is a React SPA, not WordPress. Layers 2 and 4 differ significantly from WP clients. Cloudflare is already proxying titantinyhomes.ca — this is an advantage for Layers 2 and 4.

Layer 1 — Error Tracking & Logs

Current State

None. No Sentry or equivalent. JS errors in the React SPA are invisible unless a user reports them. SiteGround server logs capture 5xx errors but are not alerted. No error aggregation.

Proposed Addition

Sentry Free tier for React (@sentry/react)

```
# In titantinyhomes repo
npm install @sentry/react
```

```
// src/main.tsx – wrap before ReactDOM.render()
import * as Sentry from "@sentry/react";

Sentry.init({
  dsn: "https://<key>@<org>.ingest.sentry.io/<project>",
  environment: import.meta.env.MODE,
  integrations: [Sentry.browserTracingIntegration()],
  tracesSampleRate: 0.1, // 10% performance tracing – sufficient for low-traffic site
});
```

```
// Wrap the app root in ErrorBoundary for caught render errors
<Sentry.ErrorBoundary fallback={<p>Something went wrong. Please refresh.</p>}>
  <App />
</Sentry.ErrorBoundary>
```

Alert config: Email on first new issue class → amglvdigital@pm.me .

Build step: Sentry source maps upload on deploy so stack traces map to readable source:

```
# .github/workflows/deploy.yml (add step after build)
- name: Upload Sentry source maps
  env:
    SENTRY_AUTH_TOKEN: ${ secrets.SENTRY_AUTH_TOKEN }
  run: npx @sentry/cli releases files ${ github.sha } upload-sourcemaps ./dist
```

Estimated effort	2h (npm install, init in main.tsx, ErrorBoundary, CI source map upload, alert config)
Risk to live site	None — monitoring layer only; ErrorBoundary adds graceful fallback rather than white-screen-of-death
Cost	\$0 (Sentry Free: 5k errors/mo)

Layer 2 — Rate Limiting

Current State

Cloudflare is already proxying titantinyhomes.ca (zone confirmed in context). No WAF rate limiting rules are currently configured. Contact form endpoint and any API calls are at full request rate.

Proposed Addition

Cloudflare WAF rules (Cloudflare Free — already have the zone)

This is the easiest hardening win across all 3 clients: Cloudflare is already in place, rules just need to be added in the dashboard.

```
# Rule 1: Rate limit contact form submissions (prevent form spam/abuse)
# Target: POST requests to the SPA (any path, POST method)
(http.request.method eq "POST") → Rate Limit: 5 req/min per IP → Block 15min

# Rule 2: Block known bad bots by user-agent
(http.user_agent contains "sqlmap" or http.user_agent contains "nikto" or
http.user_agent contains "acunetix") → Block

# Rule 3: Country-based Managed Challenge for non-CA traffic (optional, given local client)
# Note: Titan targets Ontario/Canada primarily – enable only if bot traffic becomes an issue
(not ip.geoip.country in {"CA" "US"}) → Managed Challenge
```

Also configure: Cloudflare Bot Fight Mode (free, on by default — verify it's enabled in dashboard).

Estimated effort	0.5h (add 2-3 WAF rules in CF dashboard — Cloudflare already set up)
Risk to live site	Low — POST rate limit at 5/min is generous for a tiny home lead form; adjust up if legitimate users hit it
Cost	\$0 (Cloudflare Free WAF rules, already have zone)

Layer 3 — Uptime Monitoring

Current State

None. No automated monitoring. Titan is a static SPA on SiteGround — downtime is possible after a bad deploy (FTP issue, SiteGround maintenance) but completely invisible without monitoring.

Proposed Addition

UptimeRobot Free (already setting up for all 3 clients — Titan is 1 monitor)

1. Add monitor: <https://titantinyhomes.ca> — HTTP(S), 5-min interval
2. Alert contacts: Email amglvdigital@pm.me + Slack webhook → #internal-dev
3. Response time threshold: Alert if avg response time > 3s (SPA should serve static assets fast from SiteGround CDN)

SPA-specific check: UptimeRobot checks HTTP 200 on the root URL. For a React SPA, this checks `index.html` delivery. A broken JS bundle (which would white-screen users) wouldn't trip this monitor. Sentry (Layer 1) covers that gap.

Estimated effort	0.25h (single monitor, same UptimeRobot account as other clients)
Risk to live site	None
Cost	\$0

Layer 4 — Security Hardening

Current State

React SPA = no WordPress attack surface (no wp-admin, no xmlrpc.php, no plugin ecosystem). However: SiteGround account 2FA status is unknown, GitHub repo access is not restricted, and there's no formal security review cadence. The deploy pipeline (GitHub Actions or manual FTP to SiteGround) has no secret scanning.

Proposed Addition

4a — SiteGround account 2FA

- Enable 2FA on Aiden's SiteGround account: my.siteground.com → Profile → Two-Factor Authentication
- This protects all 3 client sites simultaneously (all on same SiteGround account)

4b — GitHub repo security settings (glvcrypto/titantinyhomes)

- Settings → Branches: Protect `main` branch — require PR review before merge
- Settings → Security → Secret scanning: Enable (catches accidentally committed API keys)
- Settings → Security → Dependabot alerts: Enable (flags vulnerable npm packages)
- Review current collaborator access: only Aiden + Ben should have write access

4c — Cloudflare account 2FA

- Enable 2FA on the Cloudflare account managing titantinyhomes.ca
- A Cloudflare account compromise = ability to re-route DNS for all protected domains

4d — Dependency audit cadence (quarterly)

```
# Run quarterly in titantinyhomes repo
npm audit
npx npm-check-updates --interactive
```

Document findings in monthly retainer check-in.

4e — Remove any exposed API keys from frontend code React SPAs sometimes accidentally hardcode API keys. Audit:

```
grep -r "sk_" src/ || true
grep -r "VITE_" src/ | grep -v ".env" || true # check only non-.env files
```

Any secrets should be in `.env.local` (gitignored) or environment variables, never in committed code.

Estimated effort	1h (SiteGround 2FA, GitHub settings audit, Cloudflare 2FA, dependency scan)
Risk to live site	None — all settings changes, no code deployment
Cost	\$0

Layer 5 — Rollback Runbook

Current State

GitHub repo exists (`glvcrypto/titantinyhomes`). Deploy is SiteGround FTP (manual or GitHub Actions). No documented rollback procedure. A bad deploy would silently replace `index.html` and JS bundles — users would see the broken version immediately.

Proposed Rollback Runbook

Titan Tiny Homes — Site Rollback Runbook v1

Prerequisites:

- GitHub access to `glvcrypto/titantinyhomes` (Aiden or Ben)
- SiteGround File Manager or FTP access (Aiden)
- Cloudflare dashboard access (optional — for cache purge)

Scenario A — Deploy broke the site (white screen or broken UI)

```
# 1. Identify last known good commit
git log --oneline main | head -10

# 2. Build the previous version locally
git checkout <last-good-commit-hash>
npm install && npm run build

# 3. Upload dist/ to SiteGround via FTP
# FTP host: (SiteGround FTP credentials from Site Tools → FTP Accounts)
# Upload dist/* → public_html/titantinyhomes/ (or root, per current config)

# 4. Purge Cloudflare cache so users get the rolled-back version immediately
# Cloudflare dashboard → titantinyhomes.ca → Caching → Purge Everything
```

Scenario B — Deploy pipeline broke (GitHub Actions FTP upload failed)

1. Check GitHub Actions run log for error details
2. Re-trigger the failed workflow: GitHub → Actions → Re-run failed jobs
3. If re-run fails repeatedly: manually deploy via SiteGround File Manager (upload dist/)
4. Common cause: SiteGround FTP credentials expired → regenerate in Site Tools → FTP Accounts

Scenario C — Site down (SiteGround issue, not code)

1. Check SiteGround status: status.siteground.com
2. Check UptimeRobot alert for incident start time
3. If planned maintenance: SiteGround notifies 24h in advance via email (check amglvdigital@pm.me)
4. If unplanned: Contact SiteGround support at my.siteground.com/chat

Emergency contacts:

- SiteGround support: my.siteground.com/chat (24/7)
- Aiden: amglvdigital@pm.me / Telegram

Estimated effort	0.5h (write + verify Scenario A steps manually on staging)
Risk to live site	None — documentation only
Cost	\$0

Summary

Layer	Current State	Proposed	Effort	Risk	Cost
1. Error Tracking	None	Sentry React SDK + source maps in CI	2h	None	\$0
2. Rate Limiting	Cloudflare zone active, no rules	WAF rate limit rules (POST + bot UA + optional geo)	0.5h	Low	\$0
3. Uptime Monitoring	None	UptimeRobot (1 monitor, batch with other clients)	0.25h	None	\$0
4. Security Hardening	Unknown	SiteGround 2FA, GitHub branch protection + secret scanning, Cloudflare 2FA, dep audit	1h	None	\$0
5. Rollback Runbook	None	3-scenario runbook (deploy/pipeline/hosting)	0.5h	None	\$0
Total			~4.25h		\$0

Note on client status: Titan is currently deprioritized (not paying). Layers 3 and 4a (UptimeRobot + SiteGround 2FA) can be done in batch with other clients at near-zero marginal cost. Layers 1 and 2 require code changes — defer to next active engagement.

All items default build hardening — not billable.